Test evidence — Software Design

Document ID	SDS-EVID-001
Revision	dfa6eda
Date	2026-11-10
Status	Controlled in git

Contents

1 Purpose 3
2 Design Outputs 3
3 Dependencies 4
4 Out of scope 4

Id: `SDS-EVID-001` · Context: `test_evidence`

1 Purpose

This context takes Allure results and turns them into evidence about the spec's requirements. Its language is the **test run** (one execution of a test, at one commit, by one executor), the **executor** (the CI job or the machine that ran it), the **step** (a verification step, with its own result), the **attachment** (what a run kept) and the **tag** (a run's claim on a requirement, by its id). Allure's words (result, label, container) are read and written here and stop at its boundary.

2 Design Outputs

Component	Responsibility	Meets
Allure reader	Reads each result as a test run: its name, its test (file, class and function, from Allure's full name), its result, the requirements its `tag` labels claim, the outputs it labels, its commit; gives each requirement its status. A result it cannot read is named, never skipped.	item-03-001; realises item-04-002
pytest plugin	Labels each run of a test that claims a requirement of the spec with the commit under test and, when the working tree had uncommitted changes, marks it; records the executor and environment once per run	item-03-002, item-03-003

Allure reader. A run claims a requirement with a `tag` label naming its id: a Gherkin runner writes the scenario's tags as such labels, and a plain test writes `@allure.tag("<id>")`; a leading `@` is dropped. A run whose step failed or broke has failed, whatever its own status says. A result it cannot read is named and could hold a failed run: not JSON, not an object, not UTF-8, nested too deep, a symbolic link, a status Allure does not write, or labels that are not name and value text. Its reconciliation gives each requirement *failed* when any of its runs failed or broke, else *verified* when any passed, else *untested*; a claimed id that is no requirement is an orphan, reported when it shares a requirement id's leading letters (a mistyped `Item_01_001`), and an unrelated tag (`smoke`) is left alone.

pytest plugin. It runs inside the acceptance test run, not in `rdm`, and acts only on a test whose `@allure.tag(...)` names a requirement of the spec. It adds no requirement labels: the test's own tag is the claim. It labels the run with the commit under test and the worktree state (unknown when there is no commit), and once per run, when the run writes Allure results, records the

executor (a CI run with its run and URL, a local run with its user and host) and the environment (operating system, Python, pytest, allure-pytest and RDM versions, commit, worktree, CI details).

The release gate, the verification data, the verification report and the projection read results with the Allure reader. The plugin reads the requirement ids with the specification's reader.

3 Dependencies

Layer 3 of the dependency rule. Depends on `specification` (the plugin reads the requirement ids) and the shared kernel, both below it. Depended on by `release` (the release gate, the verification data), `publishing` (the verification report) and `graph` (the projection).

4 Out of scope

- Test-result formats other than Allure: a runner that writes another format needs an Allure adapter of its own.
- Code coverage: a unit-test measure, never acceptance evidence.